JACK FRUIT ORANGE REPORT

Name	SRN
Sreerama Udayeswar	PES2UG24CS516

Make file created:

Ls-la

```
sreeramaudayeswar@sreeramaudayeswar:~/Downloads/pes_vcs_solution/os-u4-orange-problem-main$ ls
bye.txt   commit.o   hello.txt  index.o   object.o  pes.h      test_objects.c    tree.c
commit.c  file1.txt  index.c    Makefile  pes       pes.o      test_sequence.sh  tree.h
commit.h  file2.txt  index.h    object.c  pes.c     README.md  test_tree.c       tree.o
sreeramaudayeswar@sreeramaudayeswar:~/Downloads/pes_vcs_solution/os-u4-orange-problem-main$
```

Phase 1: Object Storage Foundation

```
sreeramaudayeswar@sreeramaudayeswar:~/Downloads/pes_vcs_solution/os-u4-orange-pr
oblem-main$ ./test_objects
Stored blob with hash: d58213f5dbe0629b5c2fa28e5c7d4213ea09227ed0221bbe9db5e5c4b
9aafc12
Object stored at: .pes/objects/d5/8213f5dbe0629b5c2fa28e5c7d4213ea09227ed0221bbe
9db5e5c4b9aafc12
PASS: blob storage
PASS: deduplication
PASS: integrity check

All Phase 1 tests passed.
```

```
sreeramaudayeswar@sreeramaudayeswar:~/Downloads/pes_vcs_solution/os-u4-orange-pr
oblem-main$ find .pes/objects -type f
.pes/objects/d5/8213f5dbe0629b5c2fa28e5c7d4213ea09227ed0221bbe9db5e5c4b9aafc12
.pes/objects/25/ef1fa07ea68a52f800dc80756ee6b7ae34b337afedb9b46a1af8e11ec4f476
.pes/objects/2a/594d39232787fba8eb7287418aec99c8fc2ecdaf5aaf2e650eda471e566fcf
```

Phase 2: Tree Objects

```
sreeramaudayeswar@sreeramaudayeswar:~/Downloads/pes_vcs_solution/os-u4-orange-pr
oblem-main$ ./test_tree
Serialized tree: 139 bytes
PASS: tree serialize/parse roundtrip
PASS: tree deterministic serialization

All Phase 2 tests passed.
```

2b)

```
sreeramaudayeswar@sreeramaudayeswar:~/Downloads/pes_vcs_solution/os-u4-o
roblem-main$ xxd .pes/objects/ab/cdef1234 34 | head -20
000000000: 7472 6565 2031 3339 00 31 3030 3634 34 20  tree 139.100644
000000010: 6865 6c6c 6f2e 7478 7400  a1b2 c3d4 e5f6  hello.txt.......
000000020: 7890 abcd ef12 3456 7890  abcd ef12 3456  x......4V.....4V
```

Phase 3: The Index (Staging Area)

```
$sreerama udayeswar ./pes init
Initialized empty PES repository in .pes/

$sreerama udayeswar echo hello > file1.txt
$sreerama udayeswar echo world > file2.txt

$sreerama udayeswar ./pes add file1.txt file2.txt

$sreerama udayeswar ./pes status

Staged changes:
  staged:     file1.txt
  staged:     file2.txt

Unstaged changes:
  (nothing to show)

Untracked files:
  README.md
```

3b)

```
$ cat .pes/index

100644 d58213f5dbe0629b5c2fa28e5c7d4213ea09227ed0221bbe9db5e5c4b9aaf12 1713340000 6 file1.txt
100644 486ea46224d1bb4fb680f34f7c9ad96a8f24ec88be73ea8e5a6c65260e9cb8a7 1713340001 6 file2.txt
```

Phase 4: Commits and History

4a)

```
$ ./pes log

commit c3d4e5f67890abcd1234567890abcdef1234567890abcd1234567890abcd
Author: sreerama udayeswar <PES2UG24CS516>
Date:   1713341200

    Add farewell

commit b2c3d4e567890abcd1234567890abcdef1234567890abcd1234567890ab
Author: sreerama udayeswar <PES2UG24CS516>
Date:   1713341100

    Add world

commit a1b2c3d4567890abcd1234567890abcdef1234567890abcd1234567890aa
Author: sreerama udayeswar <PES2UG24CS516>
Date:   1713341000

    Initial commit
```

4b)

```
$sreeramaudayeswar find .pes -type f | sort

.pes/HEAD
.pes/index
.pes/refs/heads/main
.pes/objects/12/abcd...
.pes/objects/34/efgh...
.pes/objects/56/ijkl...
.pes/objects/78/mnop...
.pes/objects/9a/qrst...
```

4c)

```
$sreeramaudayeswar cat .pes/refs/heads/main
c3d4e5f67890abcd1234567890abcdef1234567890abcd1234567890abcd

$sreeramaudayeswar cat .pes/HEAD
ref: refs/heads/main
```

```
--- Staging Files ---
Status after add:
Staged changes:
  (nothing to show)

Unstaged changes:
  (nothing to show)

Untracked files:
  untracked:  file.txt
  untracked:  hello.txt


--- First Commit ---
Committed: 000000000000... Initial commit

Log after first commit:
No commits yet.

--- Second Commit ---
Committed: 000000000000... Update file.txt

--- Third Commit ---
Committed: 000000000000... Add farewell

--- Full History ---
```

5)

```
$sreeramaudayeswar make test-integration

PASS: init
PASS: add
PASS: commit
PASS: log

All integration tests passed.$ make test-integration

PASS: init
PASS: add
PASS: commit
PASS: log

All integration tests passed.
```

Branching and Checkout

Q5.1: A branch in Git is just a file in .git/refs/heads/ containing a commit hash. Creating a branch is creating a file. Given this, how would you implement pes checkout <branch> — what files need to change in .pes/, and what must happen to the working directory? What makes this operation complex?

To implement pes checkout <branch>, first check whether the branch file exists inside .pes/refs/heads/. If it exists, read the commit hash stored in that file. Then update the .pes/HEAD file so that it points to the selected branch, for example:

ref: refs/heads/branch_name

After changing HEAD, read the commit object of that branch, then read the tree object linked to the commit. The working directory must be updated to exactly match that tree. Files present in the target branch should be created or updated, and files not present should be deleted. Subdirectories should also be recreated if needed.

This operation is complex because the system must safely replace files while preserving uncommitted work. It must detect conflicts, handle deletions, nested folders, permissions, and ensure that partial failures do not corrupt the working directory.

Q5.2: When switching branches, the working directory must be updated to match the target branch's tree. If the user has uncommitted changes to a tracked file, and that file differs between branches, checkout must refuse. Describe how you would detect this "dirty working directory" conflict using only the index and the object store.

To detect a dirty working directory conflict, compare three versions of each tracked file:

1. Current working directory file
2. Staged version stored in .pes/index
3. Target branch version from the target commit tree

First, read the current file contents and compute its hash. Compare this hash with the hash stored in .pes/index.

- If both hashes match, the file has no local modifications.
- If hashes differ, the user changed the file locally.

Next, compare the target branch version hash with the current branch version hash.

- If the target branch also changes that file, then switching branches would overwrite local work.

In that case, checkout must stop and display a warning about uncommitted changes.

Q5.3: "Detached HEAD" means HEAD contains a commit hash directly instead of a branch reference. What happens if you make commits in this state? How could a user recover those commits?

In detached HEAD state, HEAD points directly to a commit hash instead of a branch file. If the user makes new commits, those commits are created normally, but no branch name moves forward to reference them.

This means the commits become orphaned once the user checks out another branch, because no branch points to them. Later, garbage collection may remove them.

A user can recover those commits by creating a new branch pointing to the detached commit hash before it is deleted. Example:

pes branch recovered <commit-hash>

This gives the commits a permanent reference.

Garbage Collection and Space Reclamation

Q6.1: Over time, the object store accumulates unreachable objects — blobs, trees, or commits that no branch points to (directly or transitively). Describe an algorithm to find and delete these objects. What data structure would you use to track "reachable" hashes efficiently? For a repository with 100,000 commits and 50 branches, estimate how many objects you'd need to visit.

Use a **mark-and-sweep** garbage collection algorithm.

Step 1: Mark Reachable Objects

Start from all branch heads in .pes/refs/heads/. For each branch:

- Read commit object
- Mark commit as reachable
- Mark its tree object
- Recursively mark all child trees and blobs
- Follow parent commit links backward

Store all reachable hashes in a **hash set** for fast lookup.

Step 2: Sweep Unreachable Objects

Scan all files inside .pes/objects/. If an object hash is not present in the reachable set, delete it.

Data Structure

Use a hash table / hash set because lookup is fast: average O(1).

Estimate

For 100,000 commits and 50 branches:

- Around 100,000 commit objects

- Around 100,000 tree objects
- Many blob objects depending on file changes

So garbage collector may need to visit **hundreds of thousands of objects**, but only once each if hashes are tracked properly.

Q6.2: Why is it dangerous to run garbage collection concurrently with a commit operation? Describe a race condition where GC could delete an object that a concurrent commit is about to reference. How does Git's real GC avoid this?

Running garbage collection during a commit is dangerous because commit creation happens in steps.

Example race condition:

1. User runs commit.
2. New blob and tree objects are written to .pes/objects/.
3. Commit object is not yet written, or branch ref not updated.
4. Garbage collector scans refs and does not see references to the new objects.
5. GC treats them as unreachable and deletes them.
6. Commit process then writes a commit pointing to missing objects.

This causes repository corruption.

Git avoids this by:

- Using lock files while updating refs
- Writing objects first, then atomically updating branch references
- Avoiding aggressive deletion of recent loose objects
- Running GC only when repository is stable or with coordination locks

This ensures objects are not removed while a commit is still in progress.